Fitted

Technical Design Specification v1.2

GPT-Orchestrated Outfit Recommendation Engine · Portfolio-Grade Architecture

1. Vision & Positioning

Fitted is a wardrobe-based fashion recommendation application. Users upload their wardrobe and receive personalised GPT-generated outfit recommendations.

> **Primary goal:** a technically impressive portfolio system demonstrating clean GPT orchestration, strict backend validation, and transparent ranking — scalable in design, not prematurely optimised.
>
> **Secondary potential:** the architecture is intentionally extensible into a real product.

> **Guiding principle:** Sampler bounds what GPT can choose → Validator enforces that bound → Ranker decides what the user sees.
> GPT never performs scoring logic. Backend always owns final authority.

- Always generate **multiple outfit options** (default K = 10)
- Support **like / dislike** interactions for lightweight personalisation
- Respect realistic outfit structures: two-piece, one-piece, layering, shoes
- Degrade gracefully when wardrobe data is sparse
- No structural rewrite required to unlock v2 features

2. Core Output Rules

- Always return a **list of outfits** — never a single outfit
- Default **K = 10**; a configurable constant, not a magic number
- Return fewer only if wardrobe constraints genuinely require it (see §12)
- Show a warning only if wardrobe is reasonably sized and valid count < K
- **No automatic GPT re-call** to fill missing slots — one generation pass per request
- Client may pass **forceRegenerate = true** to bypass the cache and trigger fresh generation

3. Session & Identity Model

Two identifiers govern caching, seeding, and repetition logic. Both must be resolved before any downstream system references them.

3.1 sessionId

- If authenticated: **sessionId = userId**. This value never expires.
- If anonymous: **sessionId = persistent cookie value**, generated on first visit. Lifetime: ~24 hours.

• Authenticated users receive **stable results indefinitely** within the same wardrobeVersion and occasion. Fresh generation only occurs when wardrobeVersion increments or occasion changes.

• sessionId is used in: cache key, RNG seed. It is never stored in outfit objects.

3.2 wardrobeVersion

• **Monotonic integer** per user. Starts at 1. Never resets.

• **The API layer increments it** on every wardrobe mutation (add item, edit item, delete item). Not the client. Not a DB trigger.

• Incrementing wardrobeVersion **intentionally invalidates the cache and changes the RNG seed**. Any wardrobe edit produces fresh outfit generation. This is a deliberate design choice.

• No wardrobe content hashing required. Hashing is permitted only for RNG seeds and cache keys.

• Stored in: user record. Logged in: generation_logs. Used in: cacheKey, session seed.

3.3 Session Seed

```
seed = hash(sessionId + wardrobeVersion + occasion + weather)
```

• Same inputs → **stable, reproducible results** across re-renders within the same session.

• Any input change → new seed → fresh generation.

• Hash function used for seed derivation only — no security requirement.

• No manual 'Refresh outfits' button in v1. Client may pass forceRegenerate = true instead (§14).

4. Data Model

4.1 WardrobeItem

```
{
  "id":           "string",
  "name":         "string",
  "type":         "top | bottom | dress | outer_layer | shoes",
  "styleTags":    ["string"],
  "colorTags":    ["string"],
  "occasionTags": ["string"],
  "warmth":       0-10,
  "material":     "string  (optional)",
  "formality":    "string  (optional)",
  "imageUrl":     "string"
}
```

• Tags are flexible strings — no enum enforcement in v1.

• **type** is set and controlled by the backend, not user-editable directly.

• Hoodie: defaults to **top** unless worn open → classify as **outer_layer**.

• **material** and **formality** are optional; forwarded to GPT for richer cohesion reasoning.

• **warmth** is stored but dormant — no filtering or scoring in v1.

4.2 Outfit (API Response Object)

```
{
  "id":             "uuid",
  "templateType":   "two_piece | one_piece",
  "items": [
    { "itemId": "abc", "role": "base_top"    },
    { "itemId": "def", "role": "base_bottom" },
    { "itemId": "ghi", "role": "outer_layer" },
    { "itemId": "jkl", "role": "shoes"       }
  ],
  "score": 3.23,
  "scoreBreakdown": {
    "baseScore":       1.0,
    "comboBoost":      2.0,
    "itemBoost":       0.73,
    "dislikePenalty":  0.5,
    "relaxedCooldown": false
  }
}
```

• Items are ordered: base roles first (base_top, base_bottom, or one_piece), then outer_layer, then shoes. **Optional roles are omitted entirely when unused — no null fields.**

• **templateType** is always explicit. The frontend never infers it from item roles.

• **scoreBreakdown** is computed at response time and returned in the API response. It is **not persisted to the database**. It may be logged in dev mode for debugging.

• **relaxedCooldown: true** means this outfit was in the cooldown buffer but re-included under fallback rules (§12). May render as a subtle UI indicator.

• Scores may be negative if dislikePenalty exceeds all positive components. This is valid — ranking is always relative, not absolute.

4.3 OutfitInteraction

```
{
  "userId":    "string",
  "outfitId":  "string",
  "itemIds":   ["id1", "id2", "id3"],
```

```
  "baseKey":   "topId:bottomId  OR  dressId",
  "fullSig":   "baseKey|outer=outerId|shoes=shoesId",
  "context":   { "occasion": "string", "weather": "string" },
  "action":    "like | dislike",
  "createdAt": "timestamp"
}
```

• **baseKey** and **fullSig** are computed by the backend at interaction time and stored so they can be queried efficiently without recomputing from itemIds.

• Only interactions are persisted. scoreBreakdown is not stored.

4.4 ItemAffinity

```
{
  "userId":        "string",
  "itemId":        "string",
  "affinityScore": number
}
```

• On outfit **like**: affinityScore += 1 for every item in the outfit.

• No negative affinity is stored. Dislike penalty is computed at scoring time from the interaction log.

5. Canonical Key Definitions

These are the two keys used throughout the system. Every section that references a key uses these exact definitions. They are never conflated or used interchangeably.

5.1 BaseKey (core silhouette key)

BaseKey identifies the core silhouette of an outfit, ignoring all optional layers. It is used for: **dislike cooldown suppression** and **diversity variant cap**.

```
If one_piece outfit:  BaseKey = dressId
If two_piece outfit:  BaseKey = topId + ":" + bottomId

Example (two_piece):  BaseKey = "abc:def"
Example (one_piece):  BaseKey = "ghi"
```

• BaseKey intentionally excludes outer_layer and shoes.

• "Unique baseKeys" means unique silhouettes — same dress with different jackets = same BaseKey.

• BaseKey is computed by the backend. It is stored on OutfitInteraction records for efficient lookup.

5.2 FullSignature (variant-level key)

FullSignature identifies a specific outfit variant including all optional slots. It is used for: **deduplication within a generation pass** and **comboBoost matching**.

```
FullSignature = BaseKey + "|outer=" + (outerId OR "none") + "|shoes=" + (shoesId OR "none")

Example (with outer, no shoes): "abc:def|outer=ghi|shoes=none"
Example (bare two-piece):       "abc:def|outer=none|shoes=none"
Example (one_piece, full):      "ghi|outer=jkl|shoes=mno"

Accessories not implemented in v1.2; reserve "|acc=" slot for future use.
```

- Same base pairing + different outer_layer = different FullSignature = different outfit.
- FullSignature is computed by the backend. Stored on OutfitInteraction for efficient lookup.
- Dislike cooldown uses **FullSignature** — outer-layer variants are treated as distinct outfits.

5.3 Key Responsibility Matrix

Key	Used for	NOT used for
`BaseKey`	Dislike cooldown suppression BaseKey variant cap (max 2 per key)	Deduplication comboBoost matching
`FullSignature`	Deduplication within generation pass comboBoost matching (like history)	Cooldown suppression Variant cap

■ These two keys must never be substituted for each other. Using FullSignature for cooldown would fail to suppress outer-layer variants of a disliked pairing. Using BaseKey for deduplication would collapse stylistically distinct variants into one.

6. Outfit Structure & Slot Normalization

6.1 Templates

Template A – two_piece	Template B – one_piece
1 × base_top 1 × base_bottom 0–1 outer_layer 0–1 shoes	1 × one_piece (dress or jumpsuit) 0–1 outer_layer 0–1 shoes

■ A dress outfit contains {dress, optional outer, optional shoes} only. Separate top/bottom items are never added to a one_piece outfit. Under-dress layering (e.g., stockings, innerwear) is explicitly out of scope for v1.2 and will require a new item role in a future version.

6.2 Role Schema

Role	Template	Required?	Notes
`base_top`	two_piece	Yes	Shirt, sweater, blouse, etc.
`base_bottom`	two_piece	Yes	Jeans, skirt, trousers, etc.
`one_piece`	one_piece	Yes	Dress or jumpsuit
`outer_layer`	both	No (0–1)	Jacket, coat, cardigan
`shoes`	both	No (0–1)	Any footwear

6.3 SlotMap (Internal Normalization Contract)

Before any validation, scoring, or key computation, every candidate outfit from GPT must be normalized into a **SlotMap** — a structured internal representation with named slots. This is a backend-only concept; it never appears in the API response.

```
SlotMap {
  dress:  itemId | null   // set if one_piece template
  top:    itemId | null   // set if two_piece template
  bottom: itemId | null   // set if two_piece template
  outer:  itemId | null   // optional for both
  shoes:  itemId | null   // optional for both
}
```

Valid SlotMaps: exactly one of the following, plus optional outer/shoes:

- dress is set, top is null, bottom is null → one_piece
- top is set, bottom is set, dress is null → two_piece

Invalid SlotMaps — reject immediately:

- dress + top or dress + bottom (mixed templates)
- Multiple tops or multiple bottoms
- No base role at all (empty outfit)
- Unknown or unrecognised role value
- Duplicate itemId appearing in more than one slot
- BaseKey and FullSignature are both computed from the SlotMap after normalization.

7. Candidate Pool Construction

> The full wardrobe is **never sent to GPT unbounded**. The sampler defines the exact set GPT may select from. The validator rejects any itemId not present in that sampled pool.

7.1 Partition by Type

- tops
- bottoms
- dresses (one_piece)
- outer_layers
- shoes

7.2 Per-Type Caps

These caps control both token cost and prompt size. If a category is at or below its cap, include all items — this ensures scarce categories are always fully represented.

Type	Default Cap	Notes
`tops`	**35**	Most wardrobe-heavy category
`bottoms`	**30**	
`dresses`	**25**	one_piece template
`outer_layers`	**20**	Optional slot
`shoes`	**25**	Optional slot

- A global constant **MAX_PROMPT_ITEMS** (default: 135, sum of all caps) acts as a hard ceiling on total items sent to GPT in a single request. Log estimated prompt item count per request.

7.3 Sampling Strategy (when over cap)

Apply a **70 / 30 split** per type. Only applied when category count *exceeds* its cap.

- **70% random** — ensures natural variety across sessions, controlled by session seed
- **30% signal-based** — items with high affinityScore, recently liked, or matching occasionTags

> ■ Cold-start: when a user has zero interaction history, signal-based selection has no data. Fall back to **100% random sampling** until at least one like or dislike exists. Log coldStartSampling = true.

7.4 Candidate Request Scaling

Compute candidateRequested using post-cap item counts (i.e. after caps and sampling are applied):

```
two_piece_base = count(sampled_tops) x count(sampled_bottoms)
```

```
one_piece_base = count(sampled_dresses)
total_base     = two_piece_base + one_piece_base

if total_base <= 5:
    candidateRequested = total_base x 3
else:
    candidateRequested = min(MAX_CANDIDATES, total_base x 3)

MAX_CANDIDATES = 40  // configurable constant
```

- No hard minimum — tiny wardrobes request proportionally fewer candidates.
- Hard ceiling of **MAX_CANDIDATES = 40** prevents runaway token usage.
- Using post-cap counts ensures candidateRequested never exceeds what the sampled pool can produce.

8. GPT Generation Strategy

8.1 Model Responsibilities

GPT IS responsible for	GPT is NOT responsible for
✓ Style cohesion	✗ Structural validation
✓ Occasion alignment	✗ Final scoring / ranking
✓ Candidate diversity	✗ Diversity caps or cooldown
✓ Initial candidate ordering	✗ JSON repair or retry logic

8.2 Prompt Guidance on Output Quality

GPT is instructed to prefer valid output over silence. Under wardrobe constraints, it may produce near-duplicate candidates. This is **intentional** — the BaseKey variant cap (§11) filters them in the ranker.

```
If uncertain, output N schema-compliant outfits anyway.
Prefer minor variation over invalid JSON.
Backend handles all rejection and filtering. Do not retry autonomously.
```

8.3 Invalid JSON Handling

1	**Run strict JSON schema validation on raw GPT output**
2	**If invalid → one repair attempt with targeted prompt**

```
Fix the following JSON to match the schema exactly.
Output JSON only — no prose, no markdown fences.
```

3 **If still invalid → fail gracefully with user-facing message**

■ Maximum one repair attempt per request. No infinite retries under any circumstance.

9. Recommendation Pipeline

Step 1 **Pool Preparation**

- Partition wardrobe by type.
- Apply per-type caps. If at or below cap, include all items (§7.2).
- Apply 70/30 sampling when over cap. Cold-start fallback: 100% random (§7.3).
- Compute candidateRequested from post-cap counts via scaling formula (§7.4).
- Derive session seed from sessionId + wardrobeVersion + occasion + weather (§3.3).

Step 2 **GPT Candidate Generation**

- Send capped wardrobe payload + occasion text.
- Request candidateRequested outfits in strict JSON (§16).
- Validate JSON schema; repair once if invalid (§8.3).

Step 3 **Normalization & Structural Validation**

- Normalize each candidate into a SlotMap (§6.3).
- Reject structurally invalid SlotMaps (§13).
- Drop exact FullSignature duplicates within this generation pass.
- Compute BaseKey and FullSignature for each valid candidate.

Step 4 **Cooldown Filter**

- Filter out candidates whose BaseKey is in the dislike cooldown buffer (§11.2).
- Set aside filtered candidates; they may be re-included under fallback rules (§12).

Step 5 **Scoring**

- Score each remaining candidate using the scoring formula (§10).
- dislikePenalty is computed from the last M interactions at this point.

Step 6 **Ranking & Diversity**

- Apply BaseKey variant cap: max 2 per BaseKey in final K (§11.1).
- Apply overused base item penalty (§11.3).
- If valid count < K, invoke fallback ladder (§12).
- Sort descending by score. Apply tie-break rules (§10.4). Return top K.

Step 7 **Response**

- Return outfits[] with scoreBreakdown.
- Include insufficientWardrobe warning if triggered (§12).
- Cache result (§14). Log asynchronously (§15).

10. Scoring & Personalization

10.1 Formula

```
score = baseScore + comboBoost + itemBoost - dislikePenalty
```

Component	Value	Condition
`baseScore`	**+1.0 (constant)**	Applied to every outfit
`comboBoost`	**+2.0**	Outfit's FullSignature matches a FullSignature from a previous like
`itemBoost`	**+0.1 × affinityScore per item**	Summed across all items; affinityScore += 1 per like
`dislikePenalty`	**–0.5 per disliked item**	Soft only — item remains eligible. Applied once per item regardless of frequency in history.

All weights are configurable constants in a single config file: BASE_SCORE, COMBO_BOOST, ITEM_BOOST_WEIGHT, DISLIKE_PENALTY, COOLDOWN_PENALTY. No magic numbers in ranking code.

- Scores may be negative if dislikePenalty exceeds all positive components (e.g. 3 disliked items, no boosts: score = 1.0 – 1.5 = –0.5). This is valid — ranking is relative. Negative scores are returned to the client as-is.

10.2 comboBoost Key

comboBoost uses the outfit's **FullSignature** (§5.2). The system checks whether this FullSignature appears in the user's like history. Occasion is not part of the match.

10.3 Defining a Disliked Item

There is no explicit item-level dislike feature. A **disliked item** is any itemId that appears in a disliked OutfitInteraction within the user's last **M interactions**.

- M is a configurable constant. Default: **M = 20**.
- Penalty is **flat –0.5 per item** regardless of how many times that item appears across the M interactions. Frequency is not accumulated.
- The disliked item set is computed at query time from the interaction log. Nothing extra is persisted.

10.4 Tie-Break Rules

When two or more candidates have equal scores in the final ranking pass, resolve deterministically in this order:

Priority	Rule
1st	Higher score wins (primary sort)
2nd	Prefer unseen BaseKey in this generation pass (favours silhouette variety)
3rd	Stable shuffle using seeded RNG seed = hash(sessionId + wardrobeVersion + occasion + weather + generationIndex)

- generationIndex is a per-request counter (0, 1, 2…) incremented as tie-breaks are applied. This ensures determinism without repeating the same order.

11. Diversity & Repetition Control

11.1 BaseKey Variant Cap

- Maximum **2 variants per BaseKey** in the final K returned to the client.
- Prefer filling unique BaseKeys first — only allow a 2nd variant of a BaseKey if no unrepresented BaseKeys remain.
- Enforced during the ranking pass (Step 6), not during validation.

11.2 Dislike Cooldown Buffer

Tracks recently disliked outfits by **BaseKey**. Suppresses the entire silhouette across all outer-layer and shoe variants.

- Buffer stores **BaseKeys** of disliked outfits. Size: **10 entries**. Eviction: **FIFO**.
- On dislike: extract BaseKey from the interaction, push to buffer, evict oldest if full.
- At ranking time (Step 4): filter out any candidate whose BaseKey is in the buffer.
- Cooldown applies to **dislike interactions only**. Shown-outfits tracking uses FullSignature (see below).

• Rationale: dislikes signal dissatisfaction with the core silhouette. Suppressing by BaseKey prevents all variants of a disliked pairing from resurfacing immediately.

11.3 Overused Base Item Penalty

• When the same base item appears in more than **OVERUSE_THRESHOLD** (default: 40%) of the candidate pool, apply a configurable soft score penalty to outfits containing it.

• Do not hard-reject. Diversity is nudged, not enforced absolutely.

• OVERUSE_THRESHOLD and the penalty amount are configurable constants.

11.4 Shown-Outfits Repetition Window

• Track the last **10 FullSignatures** shown to the user across generation passes.

• During ranking, apply a soft penalty to candidates whose FullSignature is in the window.

• This is separate from the cooldown buffer. It tracks shown outfits, not disliked outfits.

• outer_layer and shoes may repeat freely across different base pairings without penalty.

12. Fallback Ladder (Constraint Relaxation)

When the ranked candidate pool contains fewer than K valid outfits, the system relaxes constraints in a strict, deterministic order. Each step is only attempted if the previous step still yields fewer than K.

Step 1	**Normal rules**	Apply variant cap + cooldown filter + overuse penalty. Return K if possible.
Step 2	**Relax overuse penalty**	Set OVERUSE_PENALTY to zero. Re-run ranking with variant cap and cooldown still active.
Step 3	**Relax BaseKey variant cap**	Remove the 2-per-BaseKey limit. Allow unlimited variants per silhouette. Cooldown still active.
Step 4	**Relax cooldown suppression**	Re-include cooldown candidates. Score them with COOLDOWN_PENALTY = -2.0. Mark relaxedCooldown = true on each re-included outfit. Prefer silhouette diversity: mix dress and two-piece outfits if both are available.
Step 5	**Insufficient wardrobe**	Return all valid outfits found. Set insufficientWardrobe = true in the response. Include user-facing message: "Add more items to your wardrobe for better recommendations."

> **Principle:** soft penalties yield first, then diversity preferences, then memory-based suppressions. Validation rules (§13) never relax. The system returns the best available outfits given the wardrobe it has.

13. Validation Rules

> Validation is **structural only**. It runs after normalization (Step 3 of pipeline). These rules **never relax** under any circumstance, including fallback.

Reject a candidate if **any** of the following are true:

- SlotMap normalization fails for any reason (invalid role, mixed template, duplicate slot)
- one_piece role combined with base_top or base_bottom
- Duplicate itemId within the same outfit (same item in multiple slots)
- More than 1 outer_layer
- More than 1 shoes
- ItemId not present in the sampled candidate pool for this request
- Wrong base item count for the declared templateType
- templateType missing or not one of: two_piece, one_piece
- Exact duplicate FullSignature already seen in this generation pass

Invalid candidates are **silently discarded**. No silent schema repair — only one JSON format retry is permitted (§8.3).

14. Caching Strategy

Session-level caching. Cache key and policy:

```
cacheKey = hash(sessionId + wardrobeVersion + occasion + weather)
cacheTTL = 15 minutes
```

- Cache hit within TTL → return stored result immediately. No GPT call. Set cacheHit = true in response.
- Cache miss or expired TTL → generate, validate, rank, store, return.
- wardrobeVersion increment → always a cache miss (intentional, see §3.2).
- New session → always a cache miss.
- Client may pass **forceRegenerate = true** to bypass a valid cache entry and trigger fresh generation.
- **No cross-session caching. No persistent server-side cache in v1.**
- Cache serves two purposes: **predictable UX** (same outfits on re-render) and **cost control** (no GPT call on refresh within TTL).

15. Logging

Write to a dedicated **generation_logs** collection. One document per request:

```
{
  "userId":                   "string",
  "sessionId":                "string",
  "timestamp":                "ISO 8601",
  "wardrobeVersion":          number,
  "occasion":                 "string",
  "weather":                  "string",
  "candidateRequested":       number,
  "candidateReturnedRaw":     number,
  "validAfterValidation":     number,
  "returnedToClient":         number,
  "cacheHit":                 boolean,
  "forceRegenerate":          boolean,
  "invalidJson":              boolean,
  "retryUsed":                boolean,
  "validationRejectCount":    number,
  "fallbackStepReached":      number,
  "relaxedCooldownCount":     number,
  "coldStartSampling":        boolean,
  "estimatedPromptItemCount": number,
  "latencyMs":                number
}
```

• Logging is **best-effort and must never block API response latency**. Failures are silently ignored (optionally increment a dead-letter metric). Never await logging in the critical path.

• Dev mode may optionally log the full prompt and raw GPT response.

• **Key health signals post-launch:** invalidJson rate, validationRejectCount, fallbackStepReached distribution, estimatedPromptItemCount trend.

16. GPT Prompt Contract

System Prompt

```
You are a fashion outfit generator. Follow all rules exactly.

You will receive:
  - A wardrobe item list (id, type, colorTags, styleTags,
```

```
    occasionTags, warmth, material, formality)
  - A free-text occasion description

Hard rules:
  - Each outfit must be EITHER:
      (A) two_piece : exactly 1 base_top + 1 base_bottom
      (B) one_piece : exactly 1 one_piece (dress or jumpsuit)
  - Optional per outfit: 0 or 1 outer_layer,  0 or 1 shoes
  - Never mix one_piece with base_top or base_bottom
  - No duplicate items within a single outfit
  - Use ONLY item IDs from the provided wardrobe list
  - Maximise style cohesion and occasion alignment
  - Maximise diversity across the full candidate list
  - Return strictly valid JSON only -- no prose, no markdown fences
  - imageUrl is not provided and is not required
  - If uncertain, output N schema-compliant outfits anyway.
    Prefer minor variation over invalid JSON.
    Backend handles rejection. Do not retry autonomously.
```

User Prompt Template

```
Occasion: "<FREE_TEXT>"

Generate exactly <N> outfits.

Wardrobe:

<WARDROBE_JSON>

Output schema (JSON only):

{
  "outfits": [
    {
      "templateType": "two_piece | one_piece",
      "items": [
        { "itemId": "string", "role": "base_top"    },
        { "itemId": "string", "role": "base_bottom" },
        { "itemId": "string", "role": "outer_layer" },  // optional
        { "itemId": "string", "role": "shoes"       }   // optional
      ]
    }
  ]
}
```

Wardrobe payload per item: id, type, warmth, colorTags, styleTags, occasionTags, material (opt), formality (opt).

- **imageUrl excluded** — not needed for generation; reduces token count.
- outer_layer and shoes items in the schema are marked optional via comments. GPT must not include them unless the combination is stylistically justified.

17. UI Requirements

- Render outfits by mapping over **items[]** — no hardcoded top/bottom slots
- Respect item order in the response: base roles first, outer_layer and shoes last
- Read **templateType** from the outfit object — never infer it from item roles
- Support **like / dislike** gesture interaction per outfit
- Display warning message if the response includes an **insufficientWardrobe** flag
- Handle one-item base outfits gracefully (one_piece with no optional slots filled)
- scoreBreakdown may render as a dev/debug overlay; hidden in production by default
- relaxedCooldown = true may surface as a subtle UI indicator (optional)

18. Operational Safeguards

- Hard cap on GPT candidate request: **MAX_CANDIDATES = 40**
- Hard cap on total wardrobe items in prompt: **MAX_PROMPT_ITEMS = 135**
- Single JSON repair attempt only — no infinite retries
- SlotMap normalization runs before validation; validation runs before scoring
- Invalid candidates never reach the ranker
- All weights and thresholds defined as **named constants in one config file**
- Logging is asynchronous and best-effort — never blocks the API response
- wardrobeVersion is incremented by the **API layer** — not the client, not a DB trigger
- scoreBreakdown is computed at response time and **not persisted** to the database
- System degrades gracefully through fallback ladder (§12) before returning an error

19. Edge Cases

Condition	Behaviour
No tops or no bottoms	Cannot build two_piece. Use one_piece only, or proceed to fallback ladder.
No dresses / jumpsuits	Cannot build one_piece. Use two_piece only.

Wardrobe has only 1 type covered	Generate from available template only. Fallback ladder applies if count < K.
All GPT candidates fail structural validation	One JSON repair retry (§8.3). If still empty, fail gracefully with message.
Cooldown buffer full (10 entries)	Evict oldest BaseKey entry (FIFO). Buffer remains capped at 10.
New user, zero interactions	Cold-start: 100% random sampling. Log coldStartSampling = true.
forceRegenerate = true	Bypass cache. Generate fresh. Store new result in cache, resetting TTL.
wardrobeVersion increments during active session	Next request is a cache miss. New seed. Fresh generation.
Fallback reaches Step 4 (cooldown relaxed)	Re-include with COOLDOWN_PENALTY = -2.0. Set relaxedCooldown = true. Log relaxedCooldownCount.

20. Implementation Order

1	Define sessionId strategy and wardrobeVersion counter on user record (§3)
2	Update WardrobeItem schema: type enum, optional fields (§4.1)
3	Define all configurable constants in a single config file (§18)
4	Build SlotMap normalizer and BaseKey / FullSignature computers (§5, §6.3)
5	Build pool partitioner, per-type caps, cold-start sampler (§7)
6	Implement 70/30 signal-based sampling + session seed derivation (§7.3)
7	Implement candidate scaling formula using post-cap counts (§7.4)
8	Build GPT prompt builder and raw response handler (§16)
9	Implement JSON validation + single repair attempt (§8.3)
10	Implement structural validation layer using SlotMap (§13)
11	Refactor recommendation API to return outfits[] with scoreBreakdown (§4.2)
12	Implement scoring formula with configurable constants (§10)
13	Implement BaseKey variant cap and overuse penalty (§11.1, §11.3)

14	Implement dislike cooldown buffer: FIFO, size 10, BaseKey match (§11.2)
15	Implement shown-outfits repetition window: FullSignature, size 10 (§11.4)
16	Implement fallback ladder in defined order (§12)
17	Implement tie-break rules using seeded RNG (§10.4)
18	Implement like/dislike storage + affinity updates + interaction log (§4.3, §4.4)
19	Implement session-level caching with TTL and forceRegenerate (§14)
20	Refactor UI to map over items[] with templateType (§17)
21	Add async logging to generation_logs collection (§15)
22	Add warning handling, edge-case messaging, insufficientWardrobe flag (§19)

21. Non-Goals (MVP)

Explicitly excluded from v1:

✕ Accessories	✕ Underlayers (stockings, thermals)
✕ Multi-layer stacking	✕ Hard dislike / item bans
✕ Warmth-based filtering	✕ Weather engine
✕ Color harmony math	✕ Attribute-level taste learning
✕ Purchase recommendations	✕ Admin analytics UI
✕ Occasion clustering	✕ Recommendation explanations

22. Architectural Principles

> These principles are the guardrails for every implementation decision. A proposed change that violates one requires explicit justification.

• **Sampler bounds. Validator enforces. Ranker decides.** The flow is linear and non-overlapping. No step reaches back into a prior step's domain.

• **GPT generates. Backend validates. Backend ranks.** Responsibilities are independently testable. GPT drift cannot corrupt scoring.

• **No business logic inside GPT.** All rule enforcement is backend-side. GPT handles style, not structure.

• **Normalization before validation.** Every candidate is reduced to a SlotMap before any rule is applied. Structural errors surface early.

• **Keys have exactly one job.** BaseKey for cooldown and variant cap. FullSignature for deduplication and combo matching. Never swapped.

• **Personalisation is additive, not a learned model.** Simple weights, no training loop, fully debuggable.

• **Fallback is deterministic.** Constraint relaxation follows a defined ladder. Behaviour under scarcity is predictable and testable.

• **Warmth is stored but dormant.** Avoids shipping half-implemented climate logic.

• **Graceful degradation for small wardrobes.** Incomplete data never causes a crash — fallback ladder always returns something.

• **No structural rewrite required for v2.** Every component has a documented upgrade path.

23. Consistency Checklist

This checklist summarises every canonical definition and cross-cutting rule in one place. Use it during implementation and code review.

Keys

BaseKey format	dressId OR topId:bottomId (§5.1)
FullSignature format	BaseKey|outer=id-or-none|shoes=id-or-none (§5.2)
BaseKey used for	Dislike cooldown buffer + BaseKey variant cap (§11.1, §11.2)
FullSignature used for	Dedup within pass + comboBoost matching (§10.2, §13)
Keys computed from	SlotMap, after normalization (§6.3)
Keys stored on	OutfitInteraction record, computed at write time (§4.3)

Caching

Cache key	hash(sessionId + wardrobeVersion + occasion + weather) (§14)
Cache TTL	15 minutes (§14)
Cache bypass	forceRegenerate = true (§2, §14)
Cache miss triggers	wardrobeVersion increment OR new session OR TTL expiry (§3.2, §14)

Cooldown

Buffer stores	BaseKeys of disliked outfits (§11.2)
Buffer size	10 entries, FIFO eviction (§11.2)
Filter step	Step 4 of pipeline, before scoring (§9)
Relaxation penalty	COOLDOWN_PENALTY = -2.0, relaxedCooldown = true (§12)
Cooldown vs repetition window	Cooldown = dislike-triggered, BaseKey. Window = shown-outfit, FullSignature (§11.2, §11.4)

Fallback Ladder

Step 1	Normal rules (cap + cooldown + overuse penalty)
Step 2	Relax overuse penalty
Step 3	Relax BaseKey variant cap
Step 4	Relax cooldown (COOLDOWN_PENALTY + relaxedCooldown = true)
Step 5	Return fewer + insufficientWardrobe = true + user message
Never relax	Validation rules (§13)

Slot Normalization

Valid one_piece	dress set, top null, bottom null (§6.3)
Valid two_piece	top set, bottom set, dress null (§6.3)
Always rejected	Mixed templates, duplicate slots, unknown roles, empty base (§6.3)
Normalization runs	Before validation, before key computation (§9 Step 3)

Scoring

Formula	baseScore + comboBoost + itemBoost - dislikePenalty (§10.1)
comboBoost key	FullSignature match against like history (§10.2)
Dislike item source	Last M=20 interactions, flat penalty regardless of frequency (§10.3)
Negative scores	Valid and returned as-is — ranking is relative (§10.1)
All weights	Named constants in one config file (§18)

24. Future Expansion

The architecture supports all of the following without a structural rewrite:

- Warmth-based filtering triggered by a weather API
- Hard dislike / item ban system with an explicit block list
- Attribute-level taste learning (color palette, silhouette, formality)
- Occasion clustering and tag-similarity pre-filter
- Analytics dashboard (engagement rates, discard patterns, affinity trends)
- Item purchase suggestions based on detected wardrobe gaps
- Seasonal rotation scoring
- Underlayer and accessory support (new item types + new SlotMap slots)
- Admin UI for monitoring generation_logs health signals
- Accessories in FullSignature via the reserved |acc= slot

Appendix: Open Questions & Design Notes

These are not blockers for v1.2 implementation. They are pressure points, deliberate simplifications, and areas where a future decision will be needed before scaling. Each note includes a recommendation where one is warranted.

Memory & Signal Layers

A1 Memory Layer Precedence

Five overlapping memory and signal mechanisms exist: cooldown buffer, repetition window, ItemAffinity, dislikePenalty window, and cache. The spec implies a processing order through the pipeline but never states it as an explicit precedence hierarchy. The intended order is: Validation → Cooldown filter → Scoring (affinity + dislike penalty) → Variant cap → Repetition penalty → Tie-break. This should be documented as canonical so that future additions know exactly where they belong.

→ Promote the pipeline step order in §9 to the status of an authoritative precedence hierarchy. Any new mechanism added in v2 must declare which step it belongs to before it can be merged.

A2 Dislike Window vs Cooldown Buffer: Intentional Layering

Two dislike mechanisms operate simultaneously on different scopes: the cooldown buffer suppresses silhouettes via BaseKey (recent 10 dislikes), while dislikePenalty degrades individual items via the M=20 interaction window. These are not redundant — they serve different purposes. But the interaction is never stated as intentional. A silhouette can exit cooldown after 10 newer dislikes while its items still carry penalty from the M-window. This is arguably correct product behaviour: cooldown handles recency, penalty handles cumulative signal. But it should be stated explicitly.

→ Add one paragraph to §11 acknowledging this layering as intentional. State that cooldown handles short-term recency suppression and dislikePenalty handles longer-term item-level signal.

A3 AffinityScore Growth and Long-Term Scoring Stability

AffinityScore is unbounded. A frequently-liked item accumulates indefinitely: after 20 likes, itemBoost = 2.0, which matches comboBoost. After 50 likes, itemBoost = 5.0, which dominates the entire scoring formula. The personalization layer gradually shifts from outfit-level signals to item-level favouritism. This may be fine for v1, but it will cause scoring instability over time and make the system harder to debug when high-affinity items dominate every result.

→ Cap affinityScore at MAX_AFFINITY = 20 (configurable). This prevents any single item from accumulating more weight than a full comboBoost. Add to the config constants list. Decay can be added in v2 if taste evolution becomes a requirement.

A4 Cache Invalidation on Dislike Interaction

Cache TTL is 15 minutes. If a user dislikes an outfit, the interaction is logged and the cooldown buffer is updated — but the cached result is not invalidated. Within the TTL window, the user will be re-served the same outfits including the one they just disliked, unless they pass forceRegenerate = true. This is likely to feel broken to a real user. The spec does not define whether dislike interactions should invalidate the cache.

→ Dislike interactions should immediately invalidate the cache entry for that user/occasion/version. This is a one-line write on the interaction handler. Like interactions should not invalidate (positive reinforcement of stable results is fine). This is the highest-impact gap for real UX.

System Behaviour Under Edge Conditions

B1 Overuse Penalty and Wardrobe Scarcity Bias

The overuse penalty fires when a base item appears in >40% of the candidate pool. But if a user owns only two pairs of jeans, those jeans will naturally appear in ~50% of all two-piece candidates and be penalized automatically — not because they are overrepresented stylistically, but because the wardrobe is small. The penalty is designed for large wardrobes where one item is genuinely dominating. Applied to small wardrobes, it works against graceful degradation.

→ Apply overuse penalty only when total candidate pool size exceeds a minimum threshold (e.g., OVERUSE_MIN_POOL = 15). Below that threshold, skip the penalty entirely. This preserves the mechanism's intent without punishing small wardrobes.

B2 Cold-Start Signal Threshold

Cold-start is currently defined as zero interactions. But 1–3 interactions is statistically meaningless as a signal — the 30% signal-based sampler will be biased by a single lucky or accidental like. The binary definition works for v1 but may produce poor early sampling quality.

→ Define MIN_SIGNAL_THRESHOLD = 5 (configurable). Below this count, use 100% random sampling regardless. This is a one-line guard in the sampler. Log as coldStartSampling = true whenever interactionCount < MIN_SIGNAL_THRESHOLD, not only when it equals zero.

B3 Deduplication After Fallback Re-Inclusion

Exact FullSignature deduplication runs during validation (pipeline Step 3), before fallback. Fallback Step 4 re-includes cooldown candidates. If a re-included candidate somehow shares a FullSignature with an already-ranked outfit — possible if the same outfit was generated twice and one copy escaped dedup via different JSON structure — the duplicate will reach the client. Deduplication ownership is currently validation-only.

→ Add a final dedup pass as the last step of Step 6 (ranking), after all fallback re-inclusion is complete. Cost is negligible: a single set membership check per candidate. Deduplication should be both layers' responsibility: early (validation) and late (post-ranking).

Determinism & Novelty

C1 Permanent Determinism for Authenticated Users

Authenticated users have no session expiry. Their seed is stable until wardrobeVersion or occasion changes. In practice, a user who adds no new items and asks for the same occasion will receive the exact same 10 outfits every single time, indefinitely. For a coding portfolio, this is fine. For a fashion app — even a secondary one — this is the fastest path to the app feeling dead. Fashion is inherently novelty-seeking. Permanent determinism is the wrong default for this domain.

→ Add a daily re-seed mechanism: append date (YYYY-MM-DD) to the seed derivation. seed = hash(sessionId + wardrobeVersion + occasion + weather + date). This gives fresh results each day without any user action, while remaining stable within the day. forceRegenerate still available for immediate refresh.

C2 Semantic Coherence: GPT as the Only Guardrail

Validation is entirely structural. Semantic coherence — whether outfits are actually stylistically appropriate for the occasion — is delegated wholly to GPT. If GPT misbehaves (model update, prompt drift, poor context), the system has no fallback. Ten structurally valid but semantically absurd outfits would pass all validation rules and reach the user. For v1 this is a reasonable tradeoff. But it should be stated explicitly as a product choice, not left implicit.

→ Add one sentence to §22 Architectural Principles: 'Semantic coherence is intentionally delegated to GPT in v1. The backend enforces structure; GPT enforces style. A minimal semantic sanity layer (e.g., reject if >80% of results share the same base item) is a candidate for v2.'

Operational & Production Readiness

D1 Cost Observability Gap in Logging

The logging schema tracks latencyMs and estimatedPromptItemCount but not estimated token usage, model name/version, or estimated cost per request. For a portfolio system this is acceptable. For any real usage, cost blindness is the first thing that will cause a surprise. Token usage is available in the OpenAI/Anthropic API response and costs nothing to log.

→ Add to the logging schema: modelName (string), estimatedInputTokens (number), estimatedOutputTokens (number). These come directly from the API response metadata. No extra computation required. This alone makes the system production-observable.

D2 Abuse Controls and Rate Limiting

The spec assumes cooperative usage. There is no mention of rate limiting, per-user request throttling, or abuse mitigation. Each GPT call has a real cost. A single user refreshing aggressively with forceRegenerate = true could drive significant spend. This is an intentional out-of-scope decision for v1, but it should be documented as such.

→ Add to §21 Non-Goals: 'Rate limiting, per-user request throttling, and abuse mitigation are intentionally out of scope for v1. forceRegenerate should be rate-limited before any real deployment.'

D3 Warmth Field: Forward to GPT or Withhold

warmth is stored but unused by the backend. It is currently forwarded to GPT in the wardrobe payload. This means GPT may reason about it and produce warmth-influenced outfits — which is neither intended nor documented. If warmth is truly dormant, it should be stripped from the prompt payload until the weather engine is built.

→ Exclude warmth from the GPT prompt payload until the weather/warmth feature is implemented. Store it in WardrobeItem as planned, but do not forward it. Sending fields to GPT that you don't intend to use creates undocumented behaviour.

Product Philosophy

E1 Stated Priority Order

The system implicitly prioritises in this order: structural correctness → negative signal (cooldown + dislike penalty) → silhouette diversity → personalisation boosts → variant diversity → novelty. This is a product philosophy, not just an architectural consequence. It currently lives in no single place in the spec and can only be inferred by reading the pipeline and scoring sections together.

→ State this order explicitly in §22 Architectural Principles as: 'The system prioritises, in descending order: structural validity, negative user signal, silhouette diversity, personalisation, variant diversity, and novelty. Changes that reorder these priorities require explicit product sign-off.'

E2 Quantity vs Quality in GPT Output

The prompt demands exactly N outfits. GPT is never allowed to return fewer higher-confidence outfits, explain its choices, or signal uncertainty. This is a quantity-over-quality contract. It is the right tradeoff for v1 — the backend needs a consistent number of candidates to rank against. But it means GPT's confidence is invisible to the system, and a low-confidence outfit scores identically to a high-confidence one before personalisation is applied.

→ For v1, keep the strict N contract. For v2, consider allowing GPT to return a confidence field per outfit. This would let the ranker weight GPT's own uncertainty into the initial score, effectively making the backend ranker a fusion layer rather than a pure override.

These questions are grouped by theme. Items A3, A4, B1, B2, C1, and D3 are recommended for resolution before any real-user deployment, even at small scale.

End of Fitted – Technical Design Specification v1.2